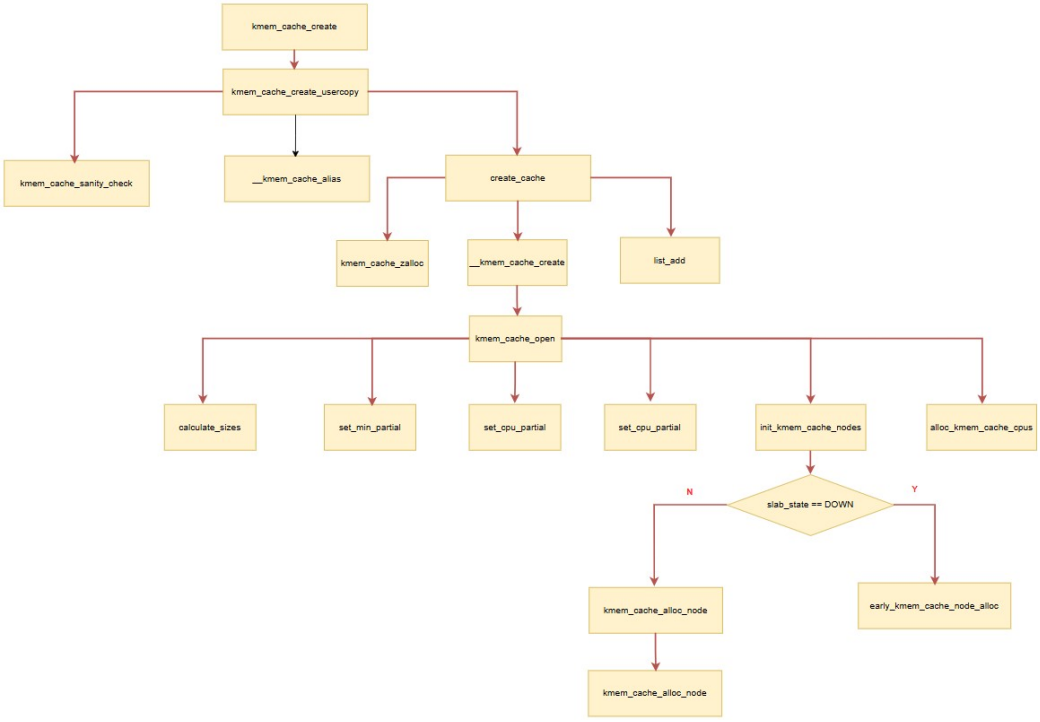
8 slab 内存池创建流程内核源码 2
8.1 创建缓存类型描述符 kmem_cache 的源码 2
8.1.1 __kmem_cache_alias 函数源码 7
8.1.2 find_mergeable 方法函数源码 8
8.1.3 calculate_alignment 方法函数源码 10
8.2 create_cache 方法函数源码 10
8.2.1 使用__kmem_cache_create 初始化 kmem_cache 对象 13
8.2.2 kmem_cache_open: slab cache 参数初始化的核心函数 14
8.2.3 calculate_sizes: slab 对象内存布局实现源码 16
8.2.4 calculate_order: 计算 slab 所需要的 page 个数 21
8.2.5 计算一个 slab 可以容纳的对象数 25
8.2.6 set_min_partial 函数 27
8.2.7 set_cpu_partial 函数 27
8.3 init_kmem_cache_nodes 方法函数源码 28
8.3.1 alloc_kmem_cache_cpus 函数源码分析 29
8.4 内核第一个 slab cache 的创建过程 30
8.4.1 slab allocator 体系的初始化 33
8.4.2 kmem_cache_node 结构的临时静态创建 37
8.4.3 将临时静态的 kmem_cache 结构变为正式的 slab cache 40
8.4.4 先创建 boot_kmem_cache_node 而不是 boot_kmem_cache 的原因 41

8 slab 内存池创建流程内核源码



8.1 创建缓存类型描述符 kmem_cache 的源码

内核中使用 **kmem_cache_create** 函数创建单类型对象的 slab 缓存池。接口中的参数，与 struct kmem_cache 结构体中一些核心属性一一对应。

```
struct kmem_cache *
kmem_cache_create(const char *name, unsigned int size, unsigned int align,
        slab_flags_t flags, void (*ctor)(void *))
{
    return kmem_cache_create_usercopy(name, size, align, flags, 0, 0,
                        ctor);
}
EXPORT_SYMBOL(kmem_cache_create);
```

```
struct kmem_cache {
    ......
    // slab cache 的管理标志位，用于设置 slab 的一些特性
    slab_flags_t flags;
```

```
    // 对应参数 size，指 slab 中对象的实际大小，不包含填充的字节数
    unsigned int object_size;

    // slab cache 的名称
    const char *name;

    // 池化对象的构造函数，用于创建 slab 对象池中的对象
    void (*ctor)(void *object);

    // 对象按照指定的 align 进行对齐
    unsigned int align;
   ......
}
```

slab cache 的整个创建过程其实是封装在 kmem_cache_create_usercopy 函数中，kmem_cache_create 直接调用了该函数，并将创建参数透传过去。

```
struct kmem_cache *
kmem_cache_create_usercopy(const char *name,
          unsigned int size, unsigned int align,
          slab_flags_t flags,
          unsigned int useroffset, unsigned int usersize,
          void (*ctor)(void *))
```

内核提供 kmem_cache_create_usercopy 函数的目的其实是为了防止 slab cache 中管理的内核核心对象被泄露，通过 useroffset 和 usersize 两个变量来指定内核对象内存布局区域中 useroffset 到 usersize 的这段内存区域可以被复制到用户空间中，其他区域则不可以。

在 Linux 内核初始化的过程中会提前为内核核心对象创建好对应的 slab cache，比如：在内核初始化函数 start_kernel 中调用 fork_init 函数为 struct task_struct 创建其所属的 slab cache —— task_struct_cachep。

在 fork_init 中就调用了 kmem_cache_create_usercopy 函数来创建 task_struct_cachep，同时指定 task_struct 对象中 useroffset 到 usersize 这段内存区域可以被复制到用户空间。例如：通过 ptrace 系统调用访问进程的 task_struct 结构时，只能访问 task_struct 对象 useroffset 到 usersize 的这段区域。

```
void __init fork_init(void)
{
    ......
    unsigned long useroffset, usersize;
    ......
    task_struct_cachep = kmem_cache_create_usercopy("task_struct",
            arch_task_struct_size, align,
            SLAB_PANIC|SLAB_ACCOUNT,
            useroffset, usersize, NULL);
    .....
```

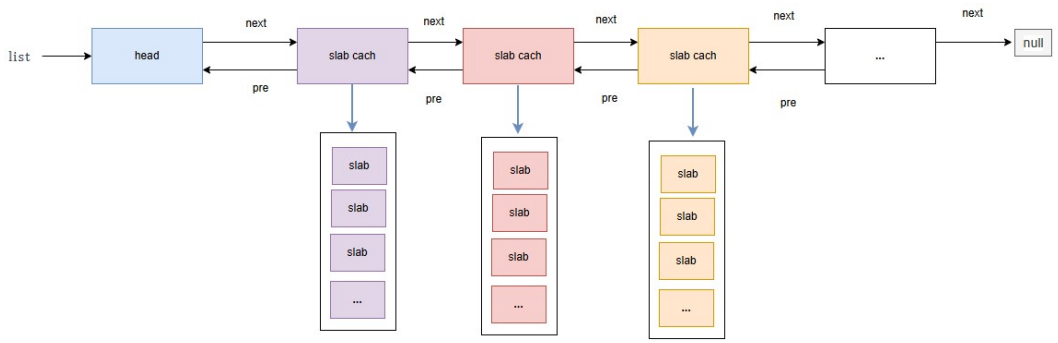
```
}
```

函数 kmem_cache_create_usercopy 内核源码如下：

```
struct kmem_cache *
kmem_cache_create_usercopy(const char *name,
          unsigned int size, unsigned int align,
          slab_flags_t flags,
          unsigned int useroffset, unsigned int usersize,
          void (*ctor)(void *))
{
    struct kmem_cache *s = NULL;
    const char *cache_name;
    int err;

#ifdef CONFIG_SLUB_DEBUG
    if (flags & SLAB_DEBUG_FLAGS)
        static_branch_enable(&slub_debug_enabled);
    if (flags & SLAB_STORE_USER)
        stack_depot_init();
#endif
  //获取 slab cache 链表的全局互斥锁
    mutex_lock(&slab_mutex);
    // 入参检查，校验 name 和 size 的有效性，防止创建过程在中断上下文中进行
    err = kmem_cache_sanity_check(name, size);
    if (err) {
        goto out_unlock;
    }
    // 检查有效的 slab flags 标记位，如果传入的 flag 是无效的，
    //则拒绝本次创建请求
    if (flags & ~SLAB_FLAGS_PERMITTED) {
        err = -EINVAL;
        goto out_unlock;
    }
     // 设置创建 slab  cache 时用到的一些标志位
    flags &= CACHE_CREATE_MASK;

    // 校验 useroffset 和 usersize 的有效性
    if (!IS_ENABLED(CONFIG_HARDENED_USERCOPY) ||
        WARN_ON(!usersize && useroffset) ||
        WARN_ON(size < usersize || size - usersize < useroffset))
        usersize = useroffset = 0;
    if (!usersize)
     // 在全局 slab cache 链表中查找与当前创建参数相匹配的 kmem_cache
     // 如果有，就不需要创建新的了，直接和已有的  slab cache  合并
```

```
     // 并且在 sys 文件系统中使用指定的 name 作为已有  slab cache  的别名
        s = __kmem_cache_alias(name, size, align, flags, ctor);
    if (s)
        goto out_unlock;
    // 在内核中为指定的 name 生成字符串常量并分配内存
    // 这里的 cache_name 就是将要创建的 slab cache 名称，
    //用于在 /proc/slabinfo 中显示
    cache_name = kstrdup_const(name, GFP_KERNEL);
    if (!cache_name) {
        err = -ENOMEM;
        goto out_unlock;
    }
    // 按照指定的参数，创建新的 slab cache
    s = create_cache(cache_name, size,
             calculate_alignment(flags, align, size),
             flags, useroffset, usersize, ctor, NULL);
    if (IS_ERR(s)) {
        err = PTR_ERR(s);
        kfree_const(cache_name);
    }
out_unlock:
    // 走到这里表示创建 slab cache 失败，释放相关的自旋锁和信号量
    mutex_unlock(&slab_mutex);
    if (err) {
        if (flags & SLAB_PANIC)
            panic("%s: Failed to create slab '%s'. Error %d\n",
                __func__, name, err);
        else {
            pr_warn("%s(%s) failed with error %d\n",
                __func__, name, err);
            dump_stack();
        }
        return NULL;
    }
    return s;
}
EXPORT_SYMBOL(kmem_cache_create_usercopy);
```

内核中使用一个全局的双向链表来串联起系统中所有的 slab cache，这里需要获取全局链表 list 的锁，防止并发对 list 进行修改。在创建 slab cache 的开始，内核为了保证整个创建过程是并发安全的，所以需要先获取 slab cache 链表的全局互斥锁。



在确保 slab cache 的整个创建过程并发安全之后，内核会首先校验 kmem_cache_create 接口函数传递进来的那些创建参数的合法有效性。比如，kmem_cache_sanity_check 函数中会确保 slab cache 的创建过程不能在中断上下文中进行，如果进程所处的上下文为中断上下文，那么内核就会返回 -EINVAL 错误停止 slab cache 的创建。因为中断处理程序是不会被内核重新调度的，这就导致处于中断上下文的操作必须是原子的，不能睡眠，不能阻塞，更不能持有锁等同步资源。而 slab cache 的创建并不是原子的，内核需要确保整个创建过程不能在中断上下文中进行。

除此之外 kmem_cache_sanity_check 函数还需要校验用户传入的 name 和 对象大小 object size 的有效性，确保 object size 小于 4M 之间。

```
#define PAGE_SHIFT      12
#define MAX_PAGE_ORDER 10
#define KMALLOC_SHIFT_MAX   (MAX_PAGE_ORDER + PAGE_SHIFT)
/* Maximum allocatable size */
#define KMALLOC_MAX_SIZE    (1UL << KMALLOC_SHIFT_MAX)


static int kmem_cache_sanity_check(const char *name, unsigned int size)
{
     // 1: 传入 slab cache 的名称不能为空
    // 2: 创建 slab cache 的过程不能处在中断上下文中
    // 3: 传入的对象大小 size 需要小于 KMALLOC_MAX_SIZE = 4M 之间
    if (!name || in_interrupt() || size > KMALLOC_MAX_SIZE) {
        pr_err("kmem_cache_create(%s) integrity check failed\n", name);
        return -EINVAL;
    }

    WARN_ON(strchr(name, ' ')); /* It confuses parsers */
    return 0;
}
```

然后，内核会校验传入的 slab cache 管理标志位 slab_flags_t 的合法性，确保 slab_flags_t 在内核规定的有效标志集合中：

```
    #define SLAB_FLAGS_PERMITTED (SLAB_CORE_FLAGS | \
                  SLAB_RED_ZONE | \
                  SLAB_POISON | \
```

```
                SLAB_STORE_USER | \
                SLAB_TRACE | \
                SLAB_CONSISTENCY_CHECKS | \
                SLAB_MEM_SPREAD | \
                SLAB_NOLEAKTRACE | \
                SLAB_RECLAIM_ACCOUNT | \
                SLAB_TEMPORARY | \
                SLAB_ACCOUNT | \
                SLAB_KMALLOC | \
                SLAB_NO_MERGE | \
                SLAB_NO_USER_FLAGS)
```

随后 flags &= CACHE_CREATE_MASK 初始化 slab_flags_t 标志位：

```
    #define CACHE_CREATE_MASK (SLAB_CORE_FLAGS | SLAB_DEBUG_FLAGS |
SLAB_CACHE_FLAGS)
```

在校验完各项创建参数的有效性之后，核会遍历系统中 slab cache 的全局链表 list，试图在系统现有 slab cache 中查找到一个各项核心参数与我们指定的创建参数贴近的 slab cache。比如，系统中存在一个 slab cache 它的各项核心参数，object size，align，slab_flags_t 和我们指定的创建参数非常贴近。

这样就可以避免重复创建新的 slab cache ，直接复用原有的 slab cache 即可，将我们指定的 name 作为原有 slab cache 的别名。如果找不到这样一个可以被复用的 slab cache，那么内核就会调用 create_cache 开始创建 slab cache 流程。

8.1.1 __kmem_cache_alias 函数源码

1. __kmem_cache_alias 函数的核心是在 find_mergeable 方法中，内核在 find_mergeable 方法里边会遍历 slab cache 的全局链表 list，查找与当前创建参数贴近可以被复用的 slab cache。

```
struct kmem_cache *
__kmem_cache_alias(const char *name, unsigned int size, unsigned int align,
           slab_flags_t flags, void (*ctor)(void *))
{
    struct kmem_cache *s;
   // 在全局 slab cache 链表中查找与当前创建参数相匹配的 slab cache
   // 如果在全局查找到一个  slab cache，它的核心参数和我们指定的创建参数很贴近
   // 那么就没必要再创建新的 slab cache 了，复用已有的 slab cache
    s = find_mergeable(size, align, flags, name, ctor);
    if (s) {
       //调用 sysfs_slab_alias 在 sys 文件系统中创建
      //一个/sys/kernel/slab/name 目录 name 就是 kmem_cache_create
      //接口函数传递过来的参数，表示要创建的 slab cache 名称。
        if (sysfs_slab_alias(s, name))
```

```
            return NULL;
        // 如果存在可复用的 kmem_cache，则将它的引用计数 + 1
        s->refcount++;
        // 采用较大的值，更新已有的 kmem_cache 相关的元数据
        s->object_size = max(s->object_size, size);
        s->inuse = max(s->inuse, ALIGN(size, sizeof(void *)));
    }
    return s;
}
```

如果通过 find_mergeable 在现有系统中所有 slab cache 中找到了一个可以复用的 slab cache，那么就不需要在创建新的了，直接返回已有的 slab cache 就可以了。在返回之前，会更新一下已有 slab cache 结构 kmem_cache 中的相关信息：

```
struct kmem_cache {
    ......
    // slab cache 的引用计数，为 0 时就可以销毁并释放内存回伙伴系统重
    int efcount;

    // 对应参数 size，指 slab 中对象的实际大小，不包含填充的字节数
    unsigned int object_size;

      // 对象的 object_size 按照 word 字长对齐之后的大小
      unsigned int inuse;
      ......
}
```

* 调用 sysfs_slab_alias 在 sys 文件系统中创建一个/sys/kernel/slab/name 目录 name 就是 kmem_cache_create 接口函数传递过来的参数，表示要创建的 slab cache 名称。

* 增加原有 slab cache 的引用计数 refcount++。

* slab cache 中的 object size 更新为我们在创建参数中指定的 object size 与原有 object size 之间的最大值。

* slab cache 中的 inuse 也是更新为原有 kmem_cache->inuse 与指定的对象 object size 与 word size 对齐之后的最大值。

8.1.2 find_mergeable 方法函数源码

find_mergeable 函数用于在 slab cache 的全局链表 list 中查找是否有可以被复用的 slab cache。一个可以被复用的 slab cache 需要满足以下 8 个条件：

1. 没有配置禁用 SLAB 合并的功能
2. 标志位 s->flags 不包含 SLAB_NEVER_MERGE 标识符
3. 该类的缓存池不存在构造函数 s->ctor，若缓存有构造函数，说明其对象需要 “定制化初始化”，无法与其他缓存合并。
4. 如果开启了 CONFIG_HARDENED_USERCOPY 条件， s->usersize 为零时。
5. kmem_cache 的引用计数 s->refcount 小于零时。内核启动阶段（bootstrap），部分

缓存的依赖资源尚未初始化，会临时将 refcount 设为负数，标记其 “启动期间不可合并”。

6. 指定对象的 object size 要小于等于已有 slab cache 中的对象 size（kmem_cache->size）。

7. 如果指定对象的 object size 与已有 kmem_cache->size 不相同，那么它们之间的差值需要再一个 word size 之内。

8. s->size 已经按照 align 对齐。

```
struct kmem_cache *find_mergeable(unsigned int size, unsigned int align,
        slab_flags_t flags, const char *name, void (*ctor)(void *))
{
    struct kmem_cache *s;

    if (slab_nomerge)
        return NULL;
    if (ctor)
        return NULL;
     // 与 word size 进行对齐
    size = ALIGN(size, sizeof(void *));
     // 根据我们指定的对齐参数 align 并结合 CPU cache line 大小，
    //计算出一个合适的对齐参数
    align = calculate_alignment(flags, align, size);
    // 对象 size 重新按照 align 进行对齐
    size = ALIGN(size, align);
     // 如果 flag 设置的是不允许合并，则停止
    flags = kmem_cache_flags(size, flags, name);
    if (flags & SLAB_NEVER_MERGE)
        return NULL;
     // 开始遍历内核中已有的 slab cache，寻找可以合并的 slab cache
    list_for_each_entry_reverse(s, &slab_caches, list) {
        if (slab_unmergeable(s))
            continue;
        // 指定对象 size 不能超过已有 slab cache 中的对象 size
         if (size > s->size)
            continue;
         // 校验指定的 flag 是否与已有 slab cache 中的 flag 一致
        if ((flags & SLAB_MERGE_SAME) != (s->flags & SLAB_MERGE_SAME))
            continue;
       //s->size 是否已经按照 align 对齐
        if ((s->size & ~(align - 1)) != s->size)
            continue;
       // 两者的 size 相差在一个 word size 之内
        if (s->size - size >= sizeof(void *))
            continue;
        return s;
    }
```

```
    return NULL;
}
```

8.1.3 calculate_alignment 方法函数源码

alculate_alignment 函数的作用时综合计算出一个合理的对齐 align。因为实际情况中，内核并不会完全按照指定的 align 进行内存对齐，而是会综合考虑 cpu 硬件 cache line 的大小，以及 word size 计算出一个合理的 align 值。内核在对 slab 对象进行内存布局的时候，会按照这个最终的 align 进行内存对齐。

```
#define ARCH_SLAB_MINALIGN __alignof__(unsigned long long)
static inline unsigned int arch_slab_minalign(void)
{
    return ARCH_SLAB_MINALIGN;
}

/*
 * Figure out what the alignment of the objects will be given a set of
 * flags, a user specified alignment and the size of the objects.
 */
static unsigned int calculate_alignment(slab_flags_t flags,
        unsigned int align, unsigned int size)
{
     // SLAB_HWCACHE_ALIGN 表示需要按照硬件 cache line 对齐
    if (flags & SLAB_HWCACHE_ALIGN) {
        unsigned int ralign;
          // 获取 cache line 大小 通常为 64 字节
        ralign = cache_line_size();
        //根据指定对齐参数 align ，对象 object size 以及 cache line 大小
        //综合计算出一个合适的对齐参数 ralign 出来
        while (size <= ralign / 2)
            ralign /= 2;
        align = max(align, ralign);
    }
    //取 align 和 slab_minalign()较大的值，
    align = max(align, arch_slab_minalign());
    return ALIGN(align, sizeof(void *));
}
```

8.2 create_cache 方法函数源码

create_cache 函数的主要任务就是为 slab cache 创建它的内核数据结构 struct

kmem_cache，并为其填充关于 struct kmem_cache 相关的属性。随后内核会为其创建 slab cache 的本地 cpu 结构 kmem_cache_cpu，每个 cpu 对应一个这样的缓存结构。然后为 slab cache 创建 NUMA 节点缓存结构 kmem_cache_node，每个 NUMA 节点对应一个。当 slab cache 的整个骨架被创建出来之后，内核会为其在 sys 文件系统中创建 /sys/kernel/slab/name 目录节点，用于详细记录该 slab cache 的运行状态以及行为信息。最后将新创建出来的 slab cache 添加到全局双向链表 list 的末尾。

```
struct kmem_cache {

   // 每个 cpu 拥有一个本地缓存，用于无锁化快速分配释放对象
    struct kmem_cache_cpu __percpu *cpu_slab;
    ......
    // slab cache 中 numa node 中的缓存，每个 node 一个
    struct kmem_cache_node *node[MAX_NUMNODES];

}
```

如下是 create_cache 函数创建 slab cache 过程的详细实现:

```
static struct kmem_cache *create_cache(const char *name,
        unsigned int object_size, unsigned int align,
        slab_flags_t flags, unsigned int useroffset,
        unsigned int usersize, void (*ctor)(void *),
        struct kmem_cache *root_cache)
{
    struct kmem_cache *s;
    int err;
    if (WARN_ON(useroffset + usersize > object_size))
        useroffset = usersize = 0;
    err = -ENOMEM;
        // 为将要创建的 slab cache 分配 kmem_cache 结构
        // kmem_cache 也是内核的一个核心数据结构，
       //同样也会被它对应的 slab cache 所管理
       // 这里就是从 kmem_cache 所属的 slab cache
        //中拿出一个 kmem_cache 对象出来
    s = kmem_cache_zalloc(kmem_cache, GFP_KERNEL);
    if (!s)
        goto out;
    // 利用指定的创建参数初始化 kmem_cache 结构
    s->name = name;
    s->size = s->object_size = object_size;
    s->align = align;
    s->ctor = ctor;
#ifdef CONFIG_HARDENED_USERCOPY
```

```
    s->useroffset = useroffset;
    s->usersize = usersize;
#endif
  //创建 slab cache 的核心函数，这里会初始化 kmem_cache 结构中的其他重要属性
  //包括创建初始化 kmem_cache_cpu 和 kmem_cache_node 结构
    err = __kmem_cache_create(s, flags);
    if (err)
        goto out_free_cache;

    // slab cache 初始状态下，引用计数为 1
    s->refcount = 1;

    // 将刚刚创建出来的 slab cache 加入到 slab cache 在内核中的全局链表管理
    list_add(&s->list, &slab_caches);
    return s;
out_free_cache:
     // 创建过程出现错误之后，释放 kmem_cache 对象
    kmem_cache_free(kmem_cache, s);
out:
    return ERR_PTR(err);
}
```

例如，task_struct，mm_struct，page，file，socket 等等一系列的内核核心数据结构都会有其专属的 slab cache 来管理。slab cache 的数据结构 struct kmem_cache 同样也属于内核的核心数据结构，它也有其专属的 slab cache 来专门管理 kmem_cache 对象的分配与释放。

内核在启动阶段，会专门为 struct kmem_cache 创建其专属的 slab cache，保存在全局变量 kmem_cache 中。

```
// 全局变量，用于专门管理 kmem_cache 对象的 slab cache
// 定义在文件：/mm/slab_common.c
struct kmem_cache *kmem_cache;
```

slab cache 的 NUMA 节点缓存 kmem_cache_node 结构也是如此，内核也会为其创建一个专属的 slab cache，保存在全局变量 kmem_cache_node 中。

```
// 全局变量，用于专门管理 kmem_cache_node 对象的 slab cache
// 定义在文件：/mm/slub.c
static struct kmem_cache *kmem_cache_node;
```

在 create_cache 函数的开始，内核会从 kmem_cache 专属的 slab cache 中申请一个 kmem_cache 对象。然后用指定的参数初始化 kmem_cache 对象。

```
s = kmem_cache_zalloc(kmem_cache, GFP_KERNEL);
```

随后会在 __kmem_cache_create 函数中近一步初始化 kmem_cache 对象的其他重要属性。比如，初始化 slab 对象的内存布局相关信息，计算 slab 所需要的物理内存页个数以及所能容纳的对象个数，创建初始化 cpu 本地缓存结构以及 NUMA 节点的缓存结构。

```
err = __kmem_cache_create(s, flags);
```

最后将刚刚创建出来的 slab cache 加入到 slab cache 在内核中的全局链表 list 中管

理。

```
list_add(&s->list, &slab_caches);
```

8.2.1 使用__kmem_cache_create 初始化 kmem_cache 对象

_kmem_cache_create 函数的主要工作就是建立 slab cache 的基本骨架，包括初始化 kmem_cache 结构中的其他重要属性，创建初始化本地 cpu 缓存结构以及 NUMA 节点缓存结构，这一部分的重要工作封装在 kmem_cache_open 函数中完成。

随后会检查内核 slab allocator 整个体系的状态，只有 slab_state = FULL 的状态才表示整个 slab allocator 体系已经在内核中建立并初始化完成了，可以正常运转了。通过 slab allocator 的状态检查之后，就是 slab cache 整个创建过程的最后一步，利用 sysfs_slab_add 为其在 sys 文件系统中创建 /sys/kernel/slab/name 目录，该目录下的文件详细记录了 slab cache 运行时的各种信息。

```
    int __kmem_cache_create(struct kmem_cache *s, slab_flags_t flags)
{
    int err;
    // 核心函数，在这里会初始化 kmem_cache 的其他重要属性
    err = kmem_cache_open(s, flags);
    if (err)
        return err;
        // 检查内核中 slab 分配器的整体体系是否已经初始化完毕，
        //只有状态是 FULL 的时候才是初始化完毕，其他的状态表示未初始化完毕。
        //在 slab  allocator 体系初始化的时候在
       //slab_sysfs_init 函数中将 slab_state 设置为 FULL
    if (slab_state <= UP)
        return 0;
     // 在 sys 文件系统中创建 /sys/kernel/slab/name 节点，
    //该目录下的文件包含了对应 slab cache 运行时的详细信息
    err = sysfs_slab_add(s);
    if (err) {
          // 出现错误则释放 kmem_cache 结构
        __kmem_cache_release(s);
        return err;
    }
    if (s->flags & SLAB_STORE_USER)
        debugfs_slab_add(s);
    return 0;
}
```

8.2.2 kmem_cache_open: slab cache 参数初始化的核心函数

kmem_cache_open 是初始化 slab cache 内核数据结构 kmem_cache 的核心函数，在这里会初始化 kmem_cache 结构中的一些重要核心参数，以及为 slab cache 创建初始化**本地 cpu 缓存结构 kmem_cache_cpu** 和 **NUMA 节点缓存结构 kmem_cache_node**。

```
static int kmem_cache_open(struct kmem_cache *s, slab_flags_t flags)
{
    s->flags = kmem_cache_flags(s->size, flags, s->name);
#ifdef CONFIG_SLAB_FREELIST_HARDENED
    s->random = get_random_long();
#endif
     // 计算 slab 中对象的整体内存布局所需要的 size
    // slab 所需最合适的内存页面大小 order，slab 中所能容纳的对象个数
    // 初始化 slab cache 中的核心参数 oo ,min,max 的值
    if (!calculate_sizes(s))
        goto error;
    if (disable_higher_order_debug) {
        /*
         * Disable debugging flags that store metadata if the min slab
         * order increased.
         */
        if (get_order(s->size) > get_order(s->object_size)) {
            s->flags &= ~DEBUG_METADATA_FLAGS;
            s->offset = 0;
            if (!calculate_sizes(s))
                goto error;
        }
#ifdef system_has_freelist_aba
    if (system_has_freelist_aba() && !(s->flags & SLAB_NO_CMPXCHG)) {
        /* Enable fast mode */
        s->flags |= __CMPXCHG_DOUBLE;
    }
#endif

    // 设置 slab cache 在 node 缓存
    //kmem_cache_node 中的 partial 列表中 slab 的最小个数 min_partial
    s->min_partial = min_t(unsigned long, MAX_PARTIAL, ilog2(s->size) / 2);
    s->min_partial = max_t(unsigned long, MIN_PARTIAL, s->min_partial);

    // 设置 slab cache 在 cpu 本地缓存的
    // partial 列表中所能容纳的最大空闲对象个数
    set_cpu_partial(s);
#ifdef CONFIG_NUMA
```

```
    s->remote_node_defrag_ratio = 1000;
#endif
    /* Initialize the pre-computed randomized freelist if slab is up */
    if (slab_state >= UP) {
        if (init_cache_random_seq(s))
            goto error;
    }
      // 为 slab cache 创建并初始化 node cache 数组
    if (!init_kmem_cache_nodes(s))
        goto error;
      // 为 slab cache 创建并初始化 cpu 本地缓存列表
    if (alloc_kmem_cache_cpus(s))
        return 0;
error:
    __kmem_cache_release(s);
    return -EINVAL;

    }
```

calculate_sizes 函数中封装了 slab 对象内存布局的全部逻辑。除了确定 slab 对象的内存布局之外，calculate_sizes 函数还会初始化 kmem_cache 的其他核心参数：

```
struct kmem_cache {
      ......
     // slab 中管理的对象大小，包含对象为了对齐所填充的字节数
    unsigned int size;
    // slab 对象池中的对象在没有被分配之前，是不关心对象里边存储的内容的。
    // 内核巧妙的利用对象占用的内存空间存储下一个空闲对象的地址。
    // offset 表示用于存储下一个空闲对象指针的位置距离对象首地址的偏移
    unsigned int offset;

        // 表示 cache 中的 slab 大小，包括 slab 所申请的页面个数，
        //以及所包含的对象个数
        // 其中低 16 位表示一个 slab 中所包含的对象总数，
        //高 16 位表示一个 slab 所占有的内存页个数。
    struct kmem_cache_order_objects oo;

      // 当按照 oo 的尺寸为 slab 申请内存时，如果内存紧张，
      //会采用 min 的尺寸为 slab 申请内存，可以容纳一个对象即可。
    struct kmem_cache_order_objects min;
    ......

}
```

完成对 kmem_cache 结构的核心属性初始化工作之后，内核紧接着会调用 set_min_partial 来设置 kmem_cache->min_partial，从而限制 slab cache 在 numa node 中

缓存的 slab 个数上限。

```
struct kmem_cache {
      ......
    //slab cache 在 numa node 中缓存的 slab 个数上限，
    //slab 个数超过该值，空闲的 empty slab 则会被回收至伙伴系统
    unsigned long min_partial;

    ......
}
```

调用 set_cpu_partial 来设置 kmem_cache->cpu_partial，从而限制 slab cache 在 cpu 本地缓存 partial 链表中空闲对象个数的上限。

```
struct kmem_cache {
      ......
    // 限定 slab cache 在每个 cpu 本地缓存 partial
    //链表中所有 slab 中空闲对象的总数
    // cpu 本地缓存 partial 链表中空闲对象的数量超过该值，
    //则会将 cpu 本地缓存 partial 链表中的所有 slab
    //转移到 numa node 缓存中。
        unsigned int cpu_partial;

    ......
}
```

调用 init_kmem_cache_nodes 函数为 slab cache 在每个 NUMA 节点中创建其所属的缓存结构 kmem_cache_node。

```
struct kmem_cache {
    ......
    struct kmem_cache_node *node[MAX_NUMNODES];
    ......
    }
```

调用 alloc_kmem_cache_cpus 函数为 slab cache 创建每个 cpu 的本地缓存结构 kmem_cache_cpu。

```
struct kmem_cache {
    ......
    struct kmem_cache_cpu __percpu *cpu_slab;
    ......
    }
```

8.2.3 calculate_sizes: slab 对象内存布局实现源码

slab 对象的内存布局的计算封装在函数 calculate_sizes 中：

```
/*
 * calculate_sizes() determines the order and the distribution of data within
 * a slab object.
```

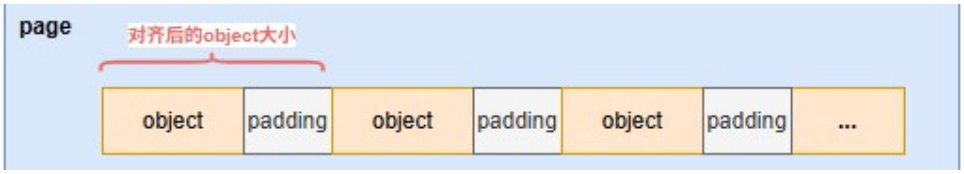
```
 */
static int (struct kmem_cache *s)
{
    slab_flags_t flags = s->flags;
    unsigned int size = s->object_size;
    unsigned int order;
    // 为了提高 cpu 访问对象的速度，
    //slab 对象的 object size 首先需要与 word size 进行对齐
    size = ALIGN(size, sizeof(void *));
#ifdef CONFIG_SLUB_DEBUG
    #ifdef CONFIG_SLUB_DEBUG
        // SLAB_POISON：对象中毒标识，是 slab 中的一个术语，
        //用于将对象所占内存填充某些特定的值，表示这块对象不同的使用状态，
        //防止非法越界访问。
        // 比如：在将对象分配出去之前，会将对象所占内存用 0x6b 填充，
        //并用 0xa5 填充 object size 区域的最后一个字节。
        // SLAB_TYPESAFE_BY_RCU：启用 RCU 锁释放 slab
    if ((flags & SLAB_POISON) && !(flags & SLAB_TYPESAFE_BY_RCU) &&
            !s->ctor)
        s->flags |= __OBJECT_POISON;
    else
        s->flags &= ~__OBJECT_POISON;
     // SLAB_RED_ZONE：表示在空闲对象前后插入
    //red zone 红色区域（填充特定字节 0xbb），防止对象溢出越界
    // size == s->object_size
    //表示对象 object size 与 word size 本来就是对齐的，并没有填充任何字节
    //这时就需要在对象 object size 内存区域的后面插入
    //一段 word size 大小的 red zone。
    // 如果对象 object size 与 word size 不是对齐的，填充了字节，
    //那么这段填充的字节恰好可以作为右侧 red zone，
    //而不需要额外分配 red zone 空间
    if ((flags & SLAB_RED_ZONE) && size == s->object_size)
        size += sizeof(void *);
#endif
    // inuse 表示 slab 中的对象实际使用的内存区域大小
    // 该值是经过与 word size 对齐之后的大小，
    //如果设置了 SLAB_RED_ZONE，则也包括红色区域大小
    s->inuse = size;
    if (slub_debug_orig_size(s) ||
        (flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)) ||
        ((flags & SLAB_RED_ZONE) && s->object_size < sizeof(void *)) ||
        s->ctor) {
        s->offset = size;
        size += sizeof(void *);
```

```
    } else {
        s->offset = ALIGN_DOWN(s->object_size / 2, sizeof(void *));
    }
#ifdef CONFIG_SLUB_DEBUG
    if (flags & SLAB_STORE_USER) {
    // SLAB_STORE_USER 表示需要跟踪对象的分配和释放信息
    // 需要再对象的末尾增加两个 struct track 结构，存储分配和释放的信息
        size += 2 * sizeof(struct track);
        /* Save the original kmalloc request size */
        if (flags & SLAB_KMALLOC)
            size += sizeof(unsigned int);
    }
#endif
    kasan_cache_create(s, &size, &s->flags);
#ifdef CONFIG_SLUB_DEBUG
    if (flags & SLAB_RED_ZONE) {
     // 在对象内存区域的左侧增加 red zone，大小为 red_left_pad
    // 防止对这块对象内存的写越界
        size += sizeof(void *);
        s->red_left_pad = sizeof(void *);
        s->red_left_pad = ALIGN(s->red_left_pad, s->align);
        size += s->red_left_pad;
    }
#endif
     // slab 从它所申请的内存页 offset 0 开始，一个接一个的存储对象
     // 调整对象的 size 保证对象之间按照指定的对齐方式 align 进行对齐
    size = ALIGN(size, s->align);
    s->size = size;
    s->reciprocal_size = reciprocal_value(size);
    // 计算 slab 所需要申请的内存页数（2 ^ order 个内存页）
    order = calculate_order(size);
    if ((int)order < 0)
        return 0;
    // 根据 slab 的 flag 设置，设置向伙伴系统申请内存时使用的 allocflags
    s->allocflags = 0;
    if (order)
        // slab 所需要的内存页多于 1 页时，则向伙伴系统申请复合页。
        s->allocflags |= __GFP_COMP;
    if (s->flags & SLAB_CACHE_DMA)
        s->allocflags |= GFP_DMA;
    if (s->flags & SLAB_CACHE_DMA32)
        s->allocflags |= GFP_DMA32;
    if (s->flags & SLAB_RECLAIM_ACCOUNT)
        s->allocflags |= __GFP_RECLAIMABLE;
```

```
    // 计算 slab cache 中的 oo，min，max 值
    // 一个 slab 到底需要多少个内存页，能够存储多少个对象
    // 低 16 为存储 slab 所能包含的对象总数，
    //高 16 为存储 slab 所需的内存页个数
    s->oo = oo_make(order, size);
    s->min = oo_make(get_order(size), size);
    return !!oo_objects(s->oo);
}
```

在内核对 slab 对象开始内存布局之前，为了提高 cpu 访问对象的速度，首先需要将 slab 对象的 object size 与 word size 进行对齐。如果 object size 与 word size 本来就是对齐的，那么内核不会做任何事情。如果不是对齐的，那么就需要在对象后面填充一些字节，达到与 word size 对齐的目的。

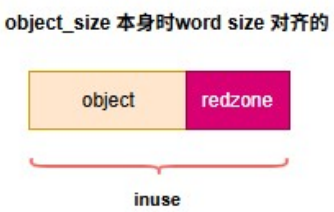
```
size = ALIGN(size, sizeof(void *));
```



如果设置了 SLAB_RED_ZONE，表示需要再对象 object 内存区域前后各插入一段 red zone 区域，目的是为了防止内存的读写越界。

如果对象 object size 与 word size 本来就是对齐的，并没有填充任何字节：size == s->object_size，那么此时就需要在对象 object size 内存区域的后面插入一段 word size 大小的 red zone。如果对象 object size 与 word size 不是对齐的，那么内核就会在 object size 区域后面填充字节达到与 word size 对齐的目的，而这段填充的字节恰好可以作为对象右侧 red zone ，而不需要额外为右侧 red zone 分配内存空间。

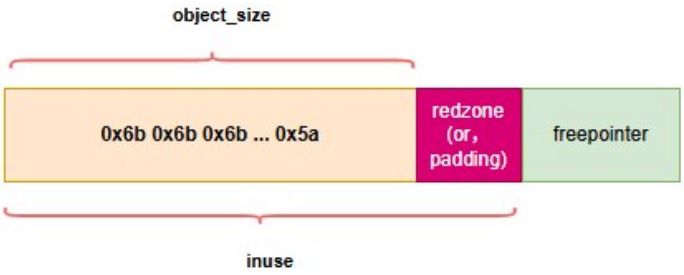
```
    if ((flags & SLAB_RED_ZONE) && size == s->object_size)
        size += sizeof(void *);
```



如果设置了 SLAB_POISON 或者开启了 RCU 或者设置了对象的构造函数，它们都会占用对象的实际内存区域 object size。比如我们设置 SLAB_POISON 之后， slab 对象的 object size 内存区域会被内核用特殊字符 0x6b 填充，并用 0xa5 填充对象 object size 内存区域的最后一个字节表示填充完毕。这样一来，用于指向下一个空闲对象的 freepointer 就没地方存放了，所以需要在当前对象内存区域的基础上再额外开辟一段 word size 大小的内存区域专门存放 freepointer。

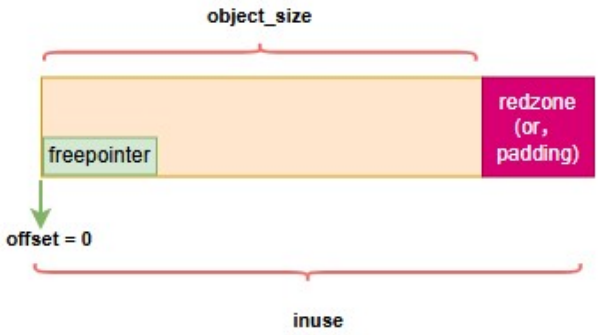
```
    if (slub_debug_orig_size(s) ||
        (flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)) ||
        ((flags & SLAB_RED_ZONE) && s->object_size < sizeof(void *)) ||
        s->ctor) {
        // offset 为 free pointer 与对象起始地址的偏移
```

```
        s->offset = size;
        size += sizeof(void *);
    } else {
        ......
    }
```



否则，除此之外，对象的 freepointer 指针就会放在对象本身内存区域 object size 中，因为在对象被分配出去之前，用户根本不会关心对象内存里到底存放的是什么。

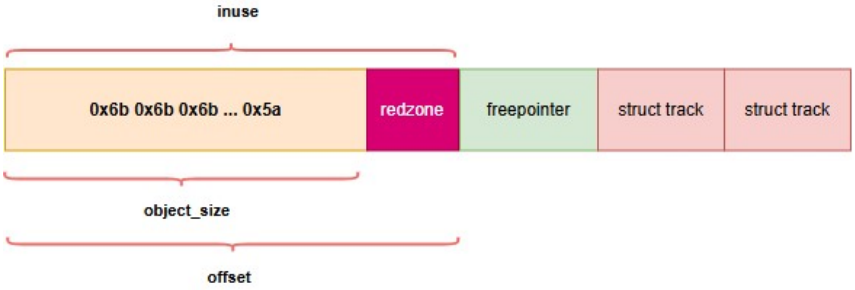
```
    if (slub_debug_orig_size(s) ||
        (flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)) ||
        ((flags & SLAB_RED_ZONE) && s->object_size < sizeof(void *)) ||
        s->ctor) {
    } else {
       // offset 为 free pointer 与对象起始地址的偏移
       s->offset = ALIGN_DOWN(s->object_size / 2, sizeof(void *));
    }
```



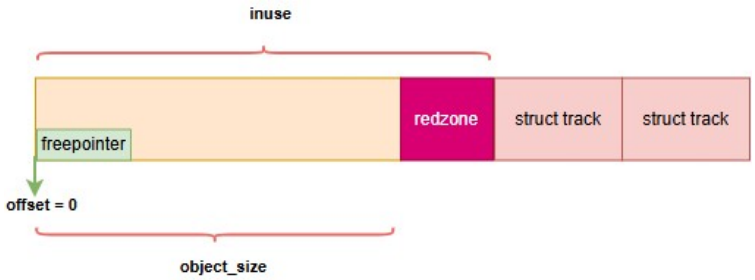
如果设置了 SLAB_STORE_USER，表示期望跟踪 slab 对象的分配与释放相关的信息，而这些跟踪信息内核使用一个 struct track 结构来存储。所以在这种情况下，内核需要在目前 slab 对象的内存区域后面额外增加两个 sizeof(struct track) 大小的区域出来，用来分别存储 slab 对象的分配和释放信息。

```
    if (flags & SLAB_STORE_USER) {
        size += 2 * sizeof(struct track);

        /* Save the original kmalloc request size */
        if (flags & SLAB_KMALLOC)
            size += sizeof(unsigned int);
    }
```

若 flag 中设置了 SLAB_POISON:

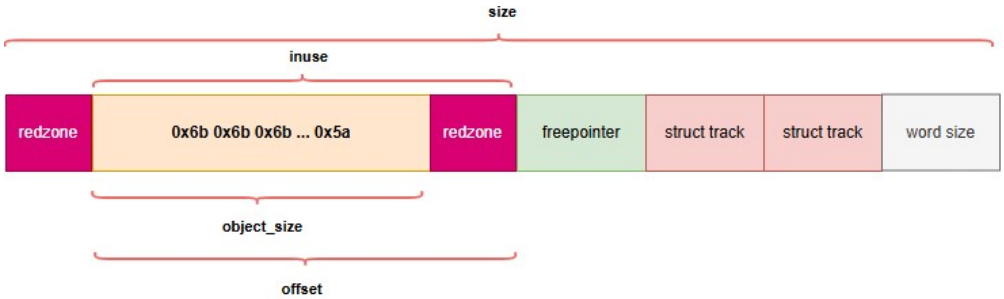


若 flag 中未设置 SLAB_POISON:

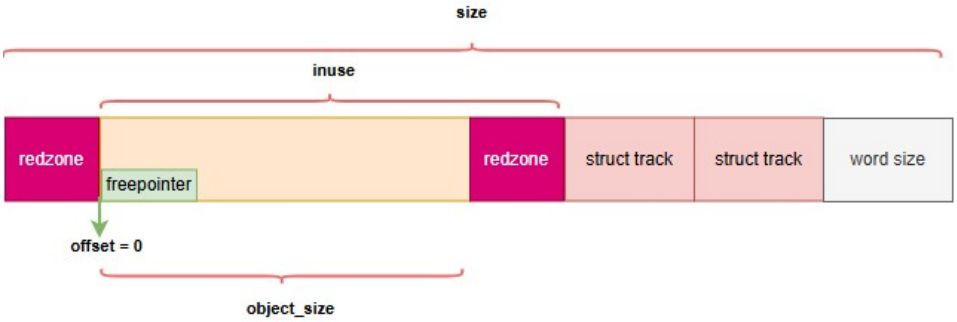


如果设置了 SLAB_RED_ZONE，最后，还需要再 slab 对象内存区域的左侧填充一段 red_left_pad 大小的内存区域作为左侧 red zone。另外还需要再 slab 对象内存区域的末尾再次填充一段 word size 大小的内存区域作为 padding 部分。

flag 中设置 SLAB_POISON 时：



flag 中未设置 SLAB_POISON 时：



8.2.4 calculate_order: 计算 slab 所需要的 page 个数

计算出 slab 对象真实占用内存大小 size 后，内核会根据这个 size，在物理内存页中划分

出一个一个的对象。内核会通过 calculate_order 函数计算出一个合理的 order 值来确定 slab 需要的物理内存页数。

```
#define PAGE_ALLOC_COSTLY_ORDER 3

static unsigned int slub_min_order;
static unsigned int slub_max_order =
    IS_ENABLED(CONFIG_SLUB_TINY) ? 1 : PAGE_ALLOC_COSTLY_ORDER;
static unsigned int slub_min_objects;
//单页允许存放的最大对象数，32767 个
#define MAX_OBJS_PER_PAGE 32767

static inline int calculate_order(unsigned int size)
{
    unsigned int order;
    unsigned int min_objects;
    unsigned int max_objects;
    unsigned int min_order;

    // 计算 slab 中可以容纳的最小对象个数
    min_objects = slub_min_objects;
    if (!min_objects) {
    // nr_cpu_ids 表示当前系统中的 cpu 个数
    // fls 可以获取参数的最高有效 bit 的位数，
    //比如 fls(0)=0，fls(1)=1，fls(4) = 3
    // 如果当前系统中有 4 个 cpu，那么 min_object 的初始值为 4*(3+1) = 16
        unsigned int nr_cpus = num_present_cpus();
        if (nr_cpus <= 1)
            nr_cpus = nr_cpu_ids;
        min_objects = 4 * (fls(nr_cpus) + 1);
    }
    // 最大对象数：slub_max_order 下能容纳的对象数，
    //至少要有 1 个
    max_objects = max(order_objects(slub_max_order, size), 1U);
    //修正最小对象数：不能超过最大对象数
    min_objects = min(min_objects, max_objects);
    //计算「最小阶数」（min_order）— 确定阶数下限
    min_order = max_t(unsigned int, slub_min_order,
              get_order(min_objects * size));

     //若 min_order 下的对象数超过单页最大对象数，调整阶数
    // 回 size * MAX_OBJS_PER_PAGE 所需要的 order 减 1
    if (order_objects(min_order, size) > MAX_OBJS_PER_PAGE)
```

```
        return get_order(size * MAX_OBJS_PER_PAGE) - 1;

    /*优先以「低浪费率（6.25%）」找阶数（内存利用率最高）
      若找不到，逐步放宽浪费率（直到 50%）；
      */
    //fraction：可接受的最大浪费率（1/fraction），
    //从 16→8→4→2（浪费率 6.25%→12.5%→25%→50%）
    for (unsigned int fraction = 16; fraction > 1; fraction /= 2) {
         // 按「最大浪费率 1/fraction」找最小阶数
        order = calc_slab_order(size, min_order, slub_max_order,
                    fraction);
        // order 不能超过 max_order，
        //否则需要降低 fraction，放宽对碎片的要求限制，重新循环计算
        if (order <= slub_max_order)
            return order;
    }
    /*兜底逻辑:放弃「多对象缓存」，仅保证单个对象能分配，若仍超过内核最大阶数
      返回 -ENOSYS（无法分配）*/
    //直接取「单个对象 size 对应的阶数」（仅能容纳 1 个对象）
    order = get_order(size);
    if (order <= MAX_PAGE_ORDER)
        return order;
    return -ENOSYS;
}
```

首先内核会计算出 slab 需要容纳对象的最小个数 min_objects，计算公式： min_objects = 4 * (fls(nr_cpu_ids) + 1)。

nr_cpu_ids 表示当前系统中的 cpu 个数

fls 获取参数二进制形式的最高有效 bit 的位数，比如 fls(0)=0，fls(1)=1，fls(4) = 3

内核规定 slab 所需要的物理内存页个数的最大值 slub_max_order 初始化为 PAGE_ALLOC_COSTLY_ORDER(3)，也就是 slab 最多只能向伙伴系统中申请 8 个内存页。根据这里的 slub_max_order 和 slab 对象的 size 通过 order_objects 函数计算出 slab 所能容纳对象的最大值。

内核通过计算出的 min_objects 可以计算出 slab 所需要的最小内存页个数，暂时称为 min_order。

随后内核会遍历 min_order 与 slub_max_order 之间的所有 order 值，直到找到满足内存碎片限制要求的一个 order。

```
    for (unsigned int fraction = 16; fraction > 1; fraction /= 2) {
        order = calc_slab_order(size, min_order, slub_max_order,
                    fraction);
        if (order <= slub_max_order)
            return order;
    }
```

内核会定义一个 fraction 变量作为 slab 内存碎片的控制系数，内核要求 slab 中内存碎

片大小不能超过 (slab 所占内存大小 / fraction)，fraction 的值越大，表示 slab 中所能容忍的内存碎片就越小。fraction 的初始值为 16。

在内核寻找最佳合适 order 的过程中，最高优先级是要将内存碎片控制在一个非常低的范围内，在这个基础之上，遍历 min_order 与 slub_max_order 之间的所有 order 值，看他们产生碎片的大小是否低于 (slab 所占内存大小 / fraction) 的要求。如果满足，那么这个 order 就是最终的计算结果，后续 slab 会根据这个 order 值向伙伴系统申请物理内存页。这个逻辑封装在 calc_slab_order 函数中。

```
static inline unsigned int calc_slab_order(unsigned int size,
        unsigned int min_order, unsigned int max_order,
        unsigned int fract_leftover)
{
    unsigned int order;
     // 从 slab 所需要的最小 order 到最大 order 之间开始遍历，
    //查找能够使 slab 碎片最小的 order 值
    for (order = min_order; order <= max_order; order++) {
       // slab 在当前 order 下，所占用的内存大小
        unsigned int slab_size = (unsigned int)PAGE_SIZE << order;
        unsigned int rem;
        // slab 的碎片大小：分配完 object 之后，所产生的碎片大小
        rem = slab_size % size;
         // 碎片大小 rem 不能超过 slab_size / fract_leftover 即符合要求
        if (rem <= slab_size / fract_leftover)
            break;
    }
    return order;
}
```

如果内核遍历完一遍 min_order 与 slub_max_order 之间的所有 order 值均不符合内存碎片限制的要求，那么内核只能尝试放宽对内存碎片的要求，将 fraction 调小一些——fraction /= 2 ，再次重新遍历所有 order。但 fraction 系数最低不能低于 1。

最极端的情况就是，无论内核怎么放宽对内存碎片的限制（fraction 系数小于 1），无论怎么放宽 slab 中容纳对象的最小个数要求，内核始终无法找到一个 order 值能够满足如此宽松的内存碎片限制条件。

最终内核的托底方案是：放弃「多对象缓存」，仅保证单个对象能分配，内核就分配容纳一个对象所需要的内存页若，仍超过内核最大阶数，则返回 -ENOSYS（无法分配）

get_order(size) 函数的逻辑计算分配一个 size 大小的对象所需要的最少内存页个数。计算逻辑如下：

size 在 [PAGE_SIZE + 1， 2^1 * PAGE_SIZE] 之间， order = 1

size 在 [2^1 * PAGE_SIZE + 1， 2^2 * PAGE_SIZE] 之间， order = 2

size 在 [2^2 * PAGE_SIZE + 1， 2^3 * PAGE_SIZE] 之间， order = 3

size 在 [2^3 * PAGE_SIZE + 1， 2^4 * PAGE_SIZE] 之间， order = 4

```
static __always_inline __attribute_const__ int get_order(unsigned long
size)
{
    if (__builtin_constant_p(size)) {
```

```
        if (!size)
            return BITS_PER_LONG - PAGE_SHIFT;

        if (size < (1UL << PAGE_SHIFT))
            return 0;
        return ilog2((size) - 1) - PAGE_SHIFT + 1;
    }
    size--;
    size >>= PAGE_SHIFT;
#if BITS_PER_LONG == 32
    return fls(size);
#else
    return fls64(size);
#endif
}
```

8.2.5 计算一个 slab 可以容纳的对象数

在计算出需要的物理页数之后，内核会根据 slab 对象占用内存的大小 size，计算出一个 slab 可以容纳的对象个数。并将这个结果保存在 kmem_cache 结构中的 oo 属性中。

```
s->oo = oo_make(order, size);
```

```
struct kmem_cache {
    ......
    // 表示 cache 中的 slab 大小，包括 slab 所申请的页面个数，
    //以及所包含的对象个数
    // 其中低 16 位表示一个 slab 中所包含的对象总数，
    //高 16 位表示一个 slab 所占有的内存页个数。
    struct kmem_cache_order_objects oo;
    ......
}
```

内核会通过 struct kmem_cache_order_objects 这样一个结构来保存 slab 所需要的物理内存页个数以及 slab 所能容纳的对象个数，其中 kmem_cache_order_objects 的高 16 位保存 slab 所需要的物理内存页个数，低 16 位保存 slab 所能容纳的对象个数。

```

#define OO_SHIFT    16
#define OO_MASK     ((1 << OO_SHIFT) - 1)

    static inline unsigned int order_objects(unsigned int order, unsigned
int size)
{
    // 根据 slab 中包含的物理内存页个数以及对象的 size，
```

```
    //计算 slab 可容纳的对象个数
      return ((unsigned int)PAGE_SIZE << order) / size;
}


static inline struct kmem_cache_order_objects oo_make(unsigned int order,
        unsigned int size)
{
    struct kmem_cache_order_objects x = {
        (order << OO_SHIFT) + order_objects(order, size)
    };

    return x;
}
static inline unsigned int oo_order(struct kmem_cache_order_objects x)
{
    // 获取高 16 位，slab 中所需要的内存页 order
    return x.x >> OO_SHIFT;
}



static inline unsigned int oo_objects(struct kmem_cache_order_objects x)
{
   // 获取低 16 位，slab 中能容纳的对象个数
     return x.x & OO_MASK;
}
```

随后内核会通过 get_order 函数来计算，容纳一个 size 大小的对象所需要的最少物理内存页个数。用这个值作为 kmem_cache 结构中的 min 属性。

```
s->min = oo_make(get_order(size), size);
```

```
struct kmem_cache {

struct kmem_cache_order_objects min;
}
```

内核在创建 slab 的时候，最开始会按照 oo 指定的尺寸来向伙伴系统申请内存页，如果内存紧张，申请内存失败。那么内核会降级采用 min 的尺寸再次向伙伴系统申请内存。也就是说 slab 中至少会包含一个对象。

8.2.6 set_min_partial 函数

set_min_partial 函数计算 slab cache 在 NUMA 节点缓存 kmem_cache_node->partial 链表中的 slab 个数上限，如果超过该上限值，空闲的 empty slab 则会被回收至伙伴系统中。

设定 min_partial 为 ilog2(s->size) / 2 需要保证 min_partial 的值在 [5,10] 的范围之内。

```
    s->min_partial = min_t(unsigned long, MAX_PARTIAL, ilog2(s->size) / 2);
    s->min_partial = max_t(unsigned long, MIN_PARTIAL, s->min_partial);
```

8.2.7 set_cpu_partial 函数

set_cpu_partial 函数设置 kmem_cache 结构的 cpu_partial 属性，该值限制了 slab cache 在 cpu 本地缓存的 partial 列表中所能容纳的最大空会从 kmem_cache_no 闲对象个数和 slab 数。同时该值也决定了当 kmem_cache_cpu->partial 链表为空时，内核 kmem_cache_node->partial 链表填充 cpu_partial_slabs / 2 个 slab 到 kmem_cache_cpu->partial 链表中。

```
static void set_cpu_partial(struct kmem_cache *s)
{
  //当配置了 CONFIG_SLUB_CPU_PARTIAL，
  //则 slab cache 的 cpu 本地缓存 kmem_cache_cpu 中包含 partial 列表
#ifdef CONFIG_SLUB_CPU_PARTIAL
    unsigned int nr_objects;
      // 判断 kmem_cache_cpu 是否包含有 partial 列表
    if (!kmem_cache_has_cpu_partial(s))
        nr_objects = 0;
    else if (s->size >= PAGE_SIZE)
        nr_objects = 6;
    else if (s->size >= 1024)
        nr_objects = 24;
    else if (s->size >= 256)
        nr_objects = 52;
    else
        nr_objects = 120;
    slub_set_cpu_partial(s, nr_objects);
#endif
}
```

```
static void slub_set_cpu_partial(struct kmem_cache *s, unsigned int
nr_objects)
{
    unsigned int nr_slabs;
```

```
    s->cpu_partial = nr_objects;
    nr_slabs = DIV_ROUND_UP(nr_objects * 2, oo_objects(s->oo));
    s->cpu_partial_slabs = nr_slabs;
}
```

8.3 init_kmem_cache_nodes 方法函数源码

slab cache 在每个 NUMA 节点中都有自己的缓存结构 kmem_cache_node。

```
struct kmem_cache {
      ......
     // slab cache 中 numa node 中的缓存，每个 node 一个
     struct kmem_cache_node *node[MAX_NUMNODES];
     ......
}
```

init_kmem_cache_nodes 函数主要逻辑是遍历所有的 NUMA 节点，并利用 struct kmem_cache_node 专属的 slab cache —— 全局变量 kmem_cache_node，分配一个 kmem_cache_node 对象，并调用 init_kmem_cache_node 对其进行初始化。

```
static int init_kmem_cache_nodes(struct kmem_cache *s)
{
    int node;
    // 遍历所有的 numa 节点，为 slab cache 创建 node cache
    for_each_node_mask(node, slab_nodes) {
        struct kmem_cache_node *n;
        if (slab_state == DOWN) {
           // 如果此时 slab allocator 体系还未建立，
          //则调用该方法分配 kmem_cache_node 结构，并初始化。
            // slab cache 的正常创建流程不会走到这个分支，
            //该分支用于在内核初始化的时候创建 kmem_cache_node 对象池使用
            early_kmem_cache_node_alloc(node);
            continue;
        }
         // 为 node cache 分配对应的 kmem_cache_node 对象
         // kmem_cache_node 对象也由它对应的 slab cache 管理
        n = kmem_cache_alloc_node(kmem_cache_node,
                        GFP_KERNEL, node);
        if (!n) {
            free_kmem_cache_nodes(s);
            return 0;
        }
          // 初始化 node cache
        init_kmem_cache_node(n);
       //初始化 slab cache 结构 kmem_cache 中的 node 数组
```

```
        s->node[node] = n;
    }
    return 1;
}
```

```
static void
init_kmem_cache_node(struct kmem_cache_node *n)
{
    n->nr_partial = 0;
    spin_lock_init(&n->list_lock);
    INIT_LIST_HEAD(&n->partial);
#ifdef CONFIG_SLUB_DEBUG
    atomic_long_set(&n->nr_slabs, 0);
    atomic_long_set(&n->total_objects, 0);
    INIT_LIST_HEAD(&n->full);
#endif
}
```

8.3.1 alloc_kmem_cache_cpus 函数源码分析

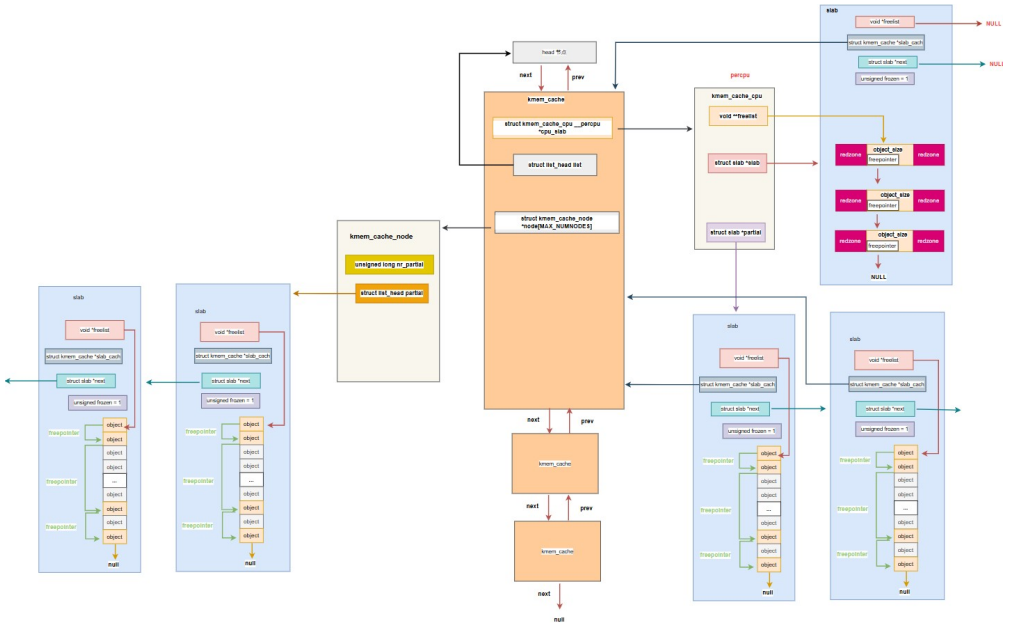
alloc_kmem_cache_cpus 函数的主要作用是为 slab cache 创建其 cpu 本地缓存结构 kmem_cache_cpu，每个 cpu 一个这样的结构，并调用 per_cpu_ptr 将创建好的 kmem_cache_cpu 结构与对应的 cpu 相关联初始化。

```
static inline int alloc_kmem_cache_cpus(struct kmem_cache *s)
{
    BUILD_BUG_ON(PERCPU_DYNAMIC_EARLY_SIZE <
            NR_KMALLOC_TYPES * KMALLOC_SHIFT_HIGH *
            sizeof(struct kmem_cache_cpu));
     // 为 slab cache 分配 cpu 本地缓存结构 kmem_cache_cpu
    // __alloc_percpu 函数在内核中专门
    //用于分配 percpu 类型的结构体（the percpu allocator）
    //kmem_cache_cpu 结构也是 percpu 类型的，
    //这里通过 __alloc_percpu 直接分配
    s->cpu_slab = __alloc_percpu(sizeof(struct kmem_cache_cpu),
                    2 * sizeof(void *));
    if (!s->cpu_slab)
        return 0;
     // 初始化 cpu 本地缓存结构 kmem_cache_cpu
    init_kmem_cache_cpus(s);
    return 1;
}
```

```
static void init_kmem_cache_cpus(struct kmem_cache *s)
```

```
{
    int cpu;
    struct kmem_cache_cpu *c;
     // 遍历所有 CPU，通过 per_cpu_ptr 将前面分配的
      //kmem_cache_cpu 结构与对应的 CPU 关联对应起来
    // 同时初始化 kmem_cache_cpu 变量里面的 tid 为其所关联 cpu 的编号
    for_each_possible_cpu(cpu) {
        c = per_cpu_ptr(s->cpu_slab, cpu);
        local_lock_init(&c->lock);
        c->tid = init_tid(cpu);
    }
}
```

至此，slab cache 的整个骨架就全部被创建出来了，最终得到的 slab cache 完整架构如下图所示：



slab cache 创建流程图：

8.4 内核第一个 slab cache 的创建过程

在 slab cache 创建的过程中需要创建两个特殊的数据结构：

1. 一个是 slab cache 自身的管理结构 struct kmem_cache。
2. 另一个是 NUMA 节点中的缓存结构 struct kmem_cache_node。

struct kmem_cache 和 struct kmem_cache_node 同样都是内核的核心数据结构，它们各自也有一个专属的 slab cache 来管理 。分别管理 kmem_cache 对象和

kmem_cache_node 对象的分配与释放。

```
// 全局变量，用于专门管理 kmem_cache 对象的 slab cache
// 定义在文件：/mm/slab_common.c
struct kmem_cache *kmem_cache;

// 全局变量，用于专门管理 kmem_cache_node 对象的 slab cache
// 定义在文件：/mm/slub.c
static struct kmem_cache *kmem_cache_node;
```

slab cache 中的 cpu 本地缓存结构 struct kmem_cache_cpu 是一个 percpu 类型的变量，由 __alloc_percpu 直接创建，并不需要一个专门的 slab cache 来管理

在 slab cache 的创建过程中，内核首先需要向 管理 struct kmem_cache 结构专属的 slab cache 申请一个 kmem_cache 对象。

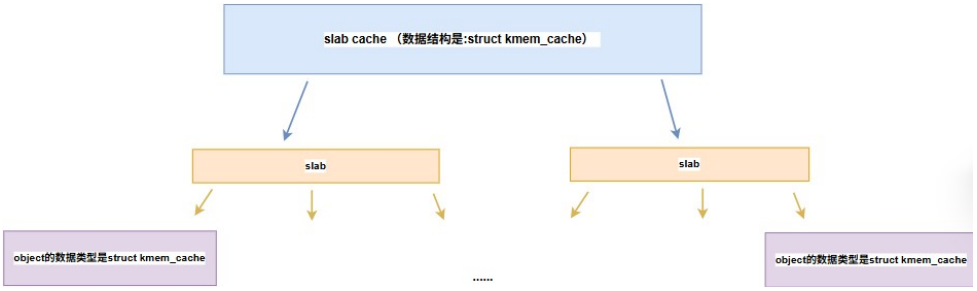
```
static struct kmem_cache *create_cache(const char *name,
        unsigned int object_size, unsigned int align,
        slab_flags_t flags, unsigned int useroffset,
        unsigned int usersize, void (*ctor)(void *),
        struct kmem_cache *root_cache)
{
        struct kmem_cache *s;

        s = kmem_cache_zalloc(kmem_cache, GFP_KERNEL);
        ......
}
```

当 kmem_cache 对象初始化完成之后，内核需要向 管理 struct kmem_cache_node 结构专属的 slab cache 申请一个 kmem_cache_node 对象，作为 slab cache 在 NUMA 节点中的缓存结构。

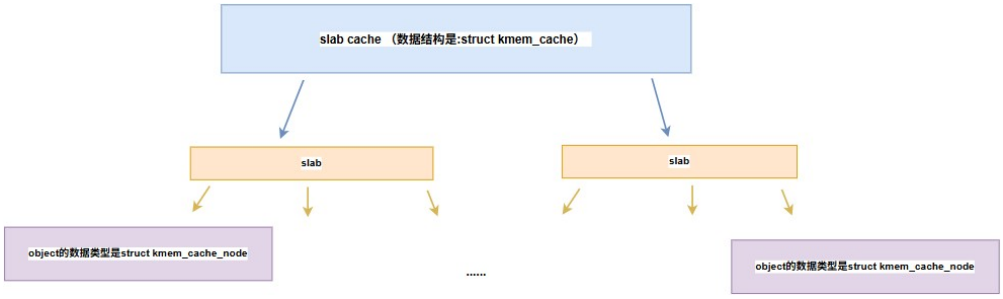
```
static int init_kmem_cache_nodes(struct kmem_cache *s)
{
    int node;

    // 遍历所有的 numa 节点，为 slab cache 创建 node cache
    for_each_node_mask(node, slab_nodes) {
        struct kmem_cache_node *n;
        if (slab_state == DOWN) {
            early_kmem_cache_node_alloc(node);
            continue;
        }
        n = kmem_cache_alloc_node(kmem_cache_node,
                        GFP_KERNEL, node);
        if (!n) {
            free_kmem_cache_nodes(s);
            return 0;
        }
```

```
        init_kmem_cache_node(n);
        s->node[node] = n;
    }
    return 1;
}
```

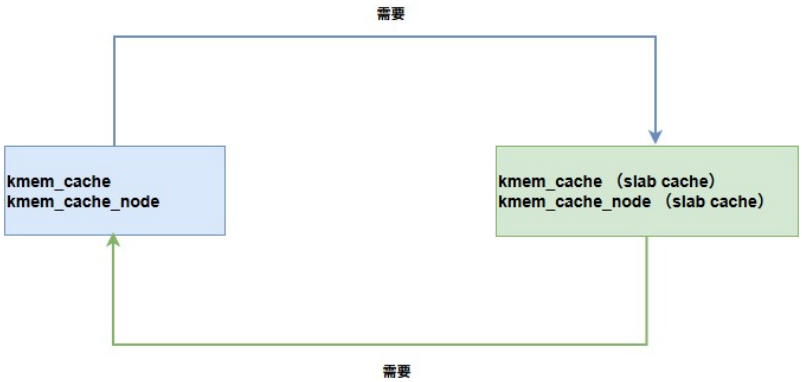
用于专门管理 struct kmem_cache 类型实体对象的 slab cache 本身又是 struct kmem_cache 类型。所以，这个 struct kmem_cache 类型的 slab cache 也可以看作是一个 struct kmem_cache 类型的实体。而分配一个这样的实体对现象,需要 struct kmem_cache 类型的 slab cache 。如下图所示，其中 slab cache 的数据类型是 struct kmem_cache；object 的数据类型也是 struct kmem_cache。



对于用于专门管理 kmem_cache_node 对象的 slab cache。可以理解为，slab cache 是 struct kmem_cache 类型(注意此处，slab cache 的数据类型仍然是 struct kmem_cache)；object 是 struct kmem_cache_node 类型。

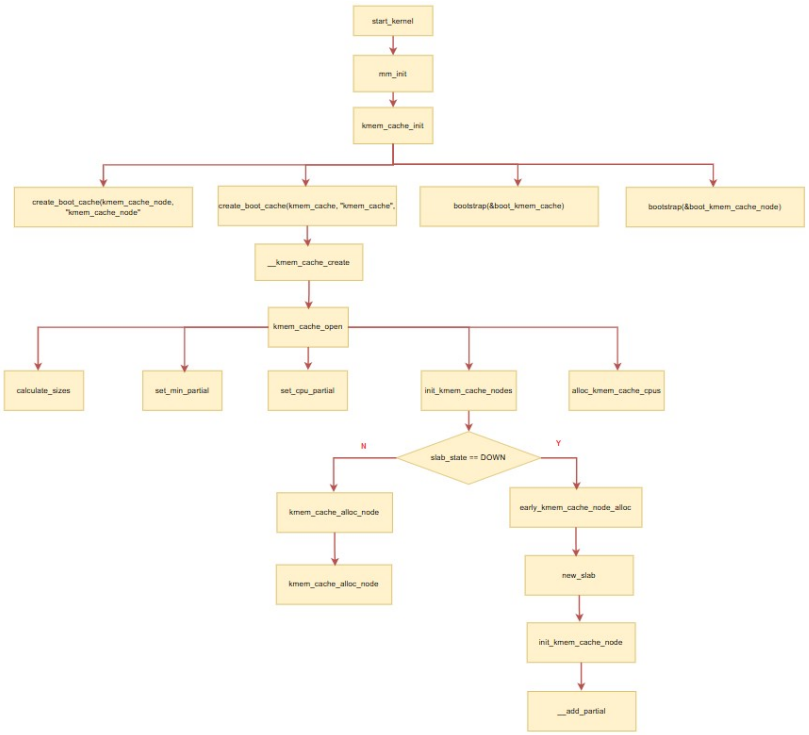


当系统中第一个 slab cache 被创建的时候，此时并没有 kmem_cache （slab cache），也没有 kmem_cache_node （slab cache）。内核是如何来解决这个问题呢？



8.4.1 slab allocator 体系的初始化

slab allocator 体系的初始化主要逻辑流程：



内核启动的核心初始化逻辑封装 \init\main.c 文件的 start_kernel 函数中，在这里会初始化内核的各个子系统，内存管理子系统的初始化工作就在这里，封装在 mm_init 函数里。在 mm_init 函数中会初始化内核的 slab allocator 体系 —— kmem_cache_init()。

```
void start_kernel(void)
{
      .......
      // 初始化内存管理子系统
      mm_core_init();
      ......

}
```

```
void __init mm_core_init(void)
{
    ......
    // 创建并初始化 slab allocator 体系
    kmem_cache_init();
    ......
}
```

为了解决开头提到的问题。内核定义两个静态的 static __initdata struct kmem_cache 结构 boot_kmem_cache，boot_kmem_cache_node ，用于在内核初始化内存管理子系统的时候临时静态地创建两个 slab cache: kmem_cache（slab cache）和 kmem_cache_node （slab cache），所需要的 struct kmem_cache 和 struct kmem_cache_node 结构。

这样一来，内核就通过这两个临时的静态 kmem_cache 结构 ：boot_kmem_cache，boot_kmem_cache_node 打破了互相依赖的关系。

当这两个临时的 boot_kmem_cache，boot_kmem_cache_node 被创建初始化之后，随后内核会通过 bootstrap 将这两个临时 slab cache 深拷贝到全局变量 kmem_cache（slab cache）和 kmem_cache_node （slab cache）中。

从此，内核就有了正式的 kmem_cache（slab cache）和 kmem_cache_node （slab cache），后续就可以按照正常流程动态地创建 slab cache 了。

slab allocator 体系的初始化过程如下：

```
void __init kmem_cache_init(void)
{
    // slab allocator 体系结构中最核心的就是
    // kmem_cache 结构和 kmem_cache_node 结构，
    //而这两个结构同时又被各自的 slab cache 所管理
    // 而现在 slab allocator 体系还未创建，
    //所以需要利用两个静态的结构来创建 kmem_cache，
    //kmem_cache_node 对象
    //这里就是定义两个 kmem_cache 类型的静态局部变量（静态结构）：
    //内核启动的时候被加载进 BSS 段中，随后会为其分配内存。
    // boot_kmem_cache 用于临时创建 kmem_cache 结构。
    // boot_kmem_cache_node 用于临时创建 kmem_cache_node 结构
    // boot_kmem_cache 和 boot_kmem_cache_node 现在只是两个空的结构，
    //需要静态的进行初始化。
    static __initdata struct kmem_cache boot_kmem_cache,
        boot_kmem_cache_node;
    int node;

    if (debug_guardpage_minorder())
        slub_max_order = 0;
    /* Print slub debugging pointers without hashing */
    if (__slub_debug_enabled())
        no_hash_pointers_enable(NULL);
    // 暂时先将这两个静态结构赋值给对应的全局变量，
    //后面会初始化这两个全局变量
    kmem_cache_node = &boot_kmem_cache_node;
    kmem_cache = &boot_kmem_cache;
    for_each_node_state(node, N_NORMAL_MEMORY)
        node_set(node, slab_nodes);

     //静态地初始化 boot_kmem_cache_node
```

```
    //从这里可以看出 slab 体系，
    //建立的第一个 slab cache 就是 kmem_cache_node(slab cache)
    create_boot_cache(kmem_cache_node, "kmem_cache_node",
        sizeof(struct kmem_cache_node), SLAB_HWCACHE_ALIGN, 0, 0);

    hotplug_memory_notifier(slab_memory_callback, SLAB_CALLBACK_PRI);
    //当 kmem_cache_node （slab cache）被创建初始化之后，
    //slab_state 变为 PARTIAL
    // 这个状态表示目前 kmem_cache_node cache 已经创建完毕，
    //可以利用它动态分配 kmem_cache_node 对象了。
    slab_state = PARTIAL;
     // 静态地初始化 boot_kmem_cache
    // 从这里可以看出 slab 体系，
    //建立的第二个 slab cache 就是 kmem_cache(slab cache)
    create_boot_cache(kmem_cache, "kmem_cache",
            offsetof(struct kmem_cache, node) +
                nr_node_ids * sizeof(struct kmem_cache_node *),
               SLAB_HWCACHE_ALIGN, 0, 0);

    // 流程到这里，两个静态的 kmem_cache 结构：
    //boot_kmem_cache，boot_kmem_cache_node 就已经初始化完毕了。
   //但是这两个静态结构只是临时的，
    //目的是为了在 slab 体系初始化阶段静态的创建
    //kmem_cache 对象和 kmem_cache_node 对象。
    // 在 bootstrap 中会将 boot_kmem_cache，
    //boot_kmem_cache_node 中的内容深拷贝到最终的
    // kmem_cache（slab cache），kmem_cache_node（slab cache）中。
    // 后面我们就可以利用这两个最终的核心结构来动态的进行 slab 创建。
    kmem_cache = bootstrap(&boot_kmem_cache);
    kmem_cache_node = bootstrap(&boot_kmem_cache_node);

            ......
}
```

初始化 slab allocator 体系的核心就是如何静态的创建和初始化这两个静态的 slab cache： boot_kmem_cache，boot_kmem_cache_node。这个核心逻辑封装在 create_boot_cache 函数中。该函数第一个参数 struct kmem_cache *s，参数 s 指向的是上面两个临时的静态的 slab cache。现在是内核初始化阶段，当前系统中并不存在一个正式完整的 slab cache。

```
void __init create_boot_cache(struct kmem_cache *s, const char *name,
        unsigned int size, slab_flags_t flags,
        unsigned int useroffset, unsigned int usersize)
{
    int err;
```

```
    unsigned int align = ARCH_KMALLOC_MINALIGN;

    //静态初始化 kmem_cache 结构的逻辑
    //挨个对 kmem_cache 结构的核心属性进行静态赋值
    s->name = name;
    s->size = s->object_size = size;
    if (is_power_of_2(size))
        align = max(align, size);
    //根据指定的对齐参数 align 以及 CPU Cache line
    //的大小计算出一个合适的 align 出来
    s->align = calculate_alignment(flags, align, size);
#ifdef CONFIG_HARDENED_USERCOPY
    s->useroffset = useroffset;
    s->usersize = usersize;
#endif
    // 这里又来到了之前介绍的创建 slab cache 的创建流程
    // 该函数是创建 slab cache 的核心函数，这里会初始化
    //kmem_cache 结构中的其他重要属性
    // 以及创建初始化 slab cache 中的 cpu 本地缓存 和 node 节点缓存结构
    err = __kmem_cache_create(s, flags);
    if (err)
        panic("Creation of kmalloc slab %s size=%u failed. Reason %d\n",
                    name, size, err);

     // 暂时不需要合并 merge，引用计数设置为 -1
    s->refcount = -1;   /* Exempt from merging for now */
}
```

这里在对静态 kmem_cache 结构进行简单初始化之后，内核又调用了 __kmem_cache_create 函数。__kmem_cache_create 函数的主要工作就是建立 slab cache 的基本骨架，包括初始化 kmem_cache 结构中的其他重要核心属性，创建初始化本地 cpu 缓存结构以及 NUMA 节点缓存结构。

当流程走到 init_kmem_cache_nodes 函数时，当前 slab_state == DOWN。当前流程正在创建初始化 boot_kmem_cache_node，所以目前内核无法利用 boot_kmem_cache_node 来动态的分配 kmem_cache_node 对象。

所以当创建初始化 boot_kmem_cache_node 的时候，流程会进入 if (slab_state == DOWN) 分支，通过 early_kmem_cache_node_alloc 函数来静态分配 kmem_cache_node 对象。

```
static int init_kmem_cache_nodes(struct kmem_cache *s)
{
    int node;
    // 遍历所有的 numa 节点，为 slub cache 创建初始化 node cache 数组
    for_each_node_mask(node, slab_nodes) {
        struct kmem_cache_node *n;
```

```
      // 当 slub 在系统启动阶段初始化时，
      //创建 kmem_cache_node cache 的时候，此时 slab_state == DOWN
     //由于此时 kmem_cache_node cache 正在创建，
      //所以无法利用 kmem_cache_node 所属的 slub cache
      //动态的分配 kmem_cache_node 对象
       //这里会通过 early_kmem_cache_node_alloc
      //函数静态的分配 kmem_cache_node 对象，并初始化。
        if (slab_state == DOWN) {
            // 创建 boot_kmem_cache_node 时会走到这个分支
            early_kmem_cache_node_alloc(node);
            continue;
        }
        //流程就会走到这里，表示此时 boot_kmem_cache_node 已经初始化，
        //这时 slab_state 为 PARTIAL，可以利用它动态的
        //分配 kmem_cache_node 对象了
         //这里的 kmem_cache_node 就是 boot_kmem_cache_node
        n = kmem_cache_alloc_node(kmem_cache_node,
                        GFP_KERNEL, node);
        if (!n) {
            free_kmem_cache_nodes(s);
            return 0;
        }
        init_kmem_cache_node(n);
        s->node[node] = n;
    }
    return 1;
}
```

在 slab allocator 体系中，第二个被创建出来的 slab cache 就 boot_kmem_cache，在创建初始化 boot_kmem_cache 的时候，slab_state 就变为了 PARTIAL，表示 kmem_cache_node 结构的专属 slab cache 已经创建出来了，可以利用它来动态分配 kmem_cache_node 对象了。

8.4.2 kmem_cache_node 结构的临时静态创建

在 slab allocator 体系中第一个被内核创建出来的 slab cache 正是 boot_kmem_cache_node，而它本身就是一个 slab cache，专门用于分配 kmem_cache_node 对象。而创建一个 slab cache 最核心的就是要为其分配 struct kmem_cache 结构（slab cache 在内核中的数据结构）还有就是 slab cache 在 NUMA 节点的缓存结构 kmem_cache_node。

针对 struct kmem_cache 结构内核已经通过定义静态结构 boot_kmem_cache_node 解

决了。

```
 static __initdata struct kmem_cache boot_kmem_cache,
       boot_kmem_cache_node;
```

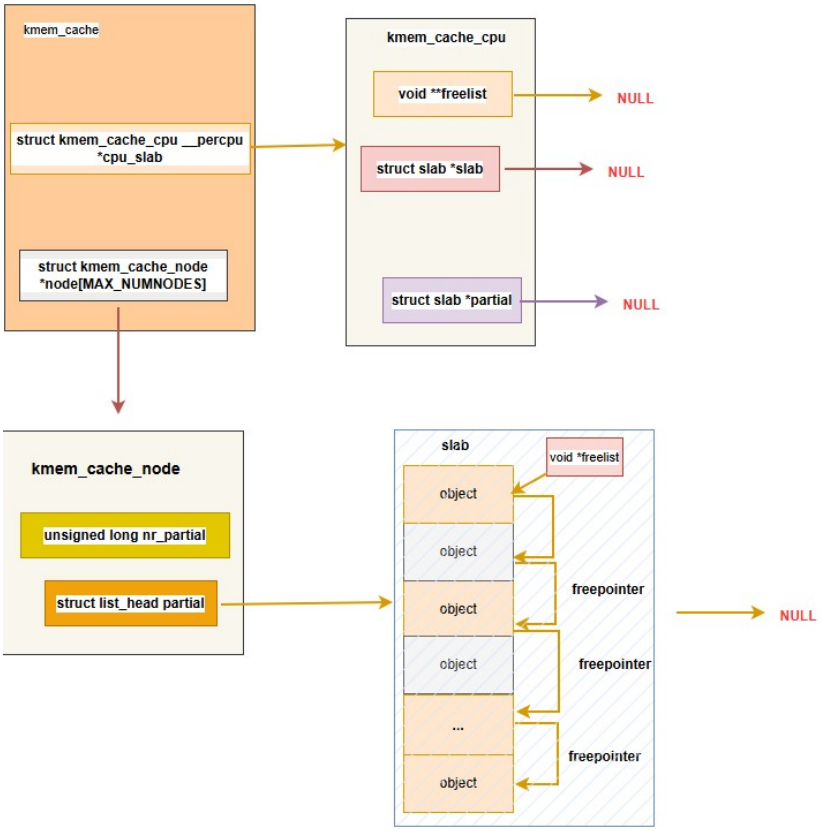
如上， boot_kmem_cache_node 的类型是 struct kmem_cache ，并不是 struct kmem_cache_node 结构类型，但它管理的是 struct kmem_cache_node 结构类型的对象。所以针对 struct kmem_cache_node 结构，内核中既没有定义这样一个静态数据结构，也没有一个 slab cache 专门管理，所以内核会通过这里的 early_kmem_cache_node_alloc 函数来创建 kmem_cache_node 对象。其，核心目标：系统启动早期为指定的 NUMA 节点初始化 kmem_cache_node 结构体。

核心步骤：分配 slab → 取出对象 → 初始化结构体 → 挂载到缓存池节点数组 → 加入 partial 链表；

```
static void early_kmem_cache_node_alloc(int node)
{
    // slab 的本质就是一个或者多个物理内存页 page，
    //这里用于指向 slab 所属的 page。
    // 如果 slab 是由多个物理页 page 组成（复合页），
    //这里指向复合页的首页
     struct slab *slab;
     // 这里主要为 boot_kmem_cache_node 初始化它的 node cache 数组
     // 这里会静态创建指定 node 节点对应的 kmem_cache_node 结构
    struct kmem_cache_node *n;

    BUG_ON(kmem_cache_node->size < sizeof(struct kmem_cache_node));

    // 此时 boot_kmem_cache_node 这个 kmem_cache 结构已经初始化好了
    // 根据 kmem_cache 结构中的 kmem_cache_order_objects oo 属性
    //指定 node 节点所属的伙伴系统申请 2^order 个内存页 page
    // 这里返回复合页的首页，目的是为 kmem_cache_node 结构分配 slab，
    //后续该 slab 会挂在 kmem_cache_node 结构中的 partial 列表中
    //kmem_cache_node：目标缓存池（专门分配 kmem_cache_node 结构体）
    slab = new_slab(kmem_cache_node, GFP_NOWAIT, node);
    BUG_ON(!slab);
    inc_slabs_node(kmem_cache_node, slab_nid(slab), slab->objects);
    if (slab_nid(slab) != node) {
        pr_err("SLUB: Unable to allocate memory from node %d\n", node);
        pr_err("SLUB: Allocating a useless per node structure in order to
be able to continue\n");
    }
     // struct slab 结构中的 freelist 指向 slab 中第一个空闲的对象
     // 此时 slab 里面的对象就是  struct kmem_cache_node 结构
    n = slab->freelist;
    BUG_ON(!n);
```

```
#ifdef CONFIG_SLUB_DEBUG
    // 根据 slab cache 中的 flag 初始化 kmem_cache_node 对象
    init_object(kmem_cache_node, n, SLUB_RED_ACTIVE);
    init_tracking(kmem_cache_node, n);
#endif
    n = kasan_slab_alloc(kmem_cache_node, n, GFP_KERNEL, false);
    slab->freelist = get_freepointer(kmem_cache_node, n);
    // 表示 slab 中已经有一个对象被使用了
    slab->inuse = 1;

    // 这里的 kmem_cache_node 指的是 boot_kmem_cache_node
    // 将初始化好的 kmem_cache_node 实例 n，
    //挂载到 kmem_cache_node 缓存池的 node[MAX_NUMNODES]
    //数组中对应目标节点的位置 —— 这是该函数的核心目标：
    //让缓存池“知道”目标节点的 kmem_cache_node 结构体在哪里
    kmem_cache_node->node[node] = n;
    // 初始化 node 节点对应的 kmem_cache_node 结构
    init_kmem_cache_node(n);
    // kmem_cache_node 结构中的 nr_slabs
    //计数加 1，total_objects 加 page->objects
    inc_slabs_node(kmem_cache_node, node, slab->objects);
    // 将新创建出来的 slab （page 表示），
    //添加到对象 n （kmem_cache_node 结构）中的 partial 列表头部
    __add_partial(n, slab, DEACTIVATE_TO_HEAD);
}
```

boot_kmem_cache_node 被初始化之后，它的整个结构如下图所示：



8.4.3 将临时静态的 kmem_cache 结构变为正式的 slab cache

到这里 boot_kmem_cache，boot_kmem_cache_node 这两个静态结构就已经被初始化好了，现在内核就可以通过他们来动态的创建 kmem_cache 对象和 kmem_cache_node 对象了。

但是这里的 boot_kmem_cache 和 boot_kmem_cache_node 只是临时的 kmem_cache 结构，其目的是在 slab allocator 体系初始化的时候用于静态创建 kmem_cache（slab cache），kmem_cache_node （slab cache）。

需要创建两个最终的全局 kmem_cache 结构，然后将这两个静态临时结构深拷贝到最终的全局 kmem_cache 结构中。

```
    static struct kmem_cache * __init bootstrap(struct kmem_cache
*static_cache)
{
    int node;
    // kmem_cache 指向专门管理 kmem_cache 对象的 slab 缓存池
    // 该 slab 缓存池 现在已经全部初始化完毕，
    //可以利用它动态的分配最终的 kmem_cache 对象
    struct kmem_cache *s = kmem_cache_zalloc(kmem_cache, GFP_NOWAIT);
    struct kmem_cache_node *n;
    // 将静态的 kmem_cache 对象，
```

```
    //比如：boot_kmem_cache，boot_kmem_cache_node
    // 深拷贝到最终的 kmem_cache 对象 s 中
    memcpy(s, static_cache, kmem_cache->object_size);
     // 释放本地 cpu 缓存的 slab
    __flush_cpu_slab(s, smp_processor_id());
    //遍历 node cache 数组，修正 kmem_cache_node 结构中
    //partial 链表中包含的 slab 结构体中的 slab_cache 指针
    //使其指向最终的 kmem_cache 结构，
    //之前初始化 boot_kmem_cache_node 流程中创建 kmem_cache_node 对象时，
    //在 create_boot_cache 中指向的是静态 kmem_cache
    //结构 boot_kmem_cache_node，这里需要修正
    for_each_kmem_cache_node(s, node, n) {
        struct slab *p;
        list_for_each_entry(p, &n->partial, slab_list)
            p->slab_cache = s;
#ifdef CONFIG_SLUB_DEBUG
        list_for_each_entry(p, &n->full, slab_list)
            p->slab_cache = s;
#endif
    }
    // 将最终的 kmem_cache 结构加入到全局 slab cache 链表中
    list_add(&s->list, &slab_caches);
    return s;
}
```

8.4.4 先创建 boot_kmem_cache_node 而不是 boot_kmem_cache 的原因

kmem_cache 和 kmem_cache_node 数据结构是 slab cache 创建要依赖两个核心结构。其中 kmem_cache 结构是 slab cache 在内核中的数据结构，同样也需要被一个专门的 slab cache 所管理，但是在内核初始化阶段 slab 体系还未建立，所以内核通过定义两个局部静态变量来解决 kmem_cache 结构的创建问题。

随后内核会在 calculate_size 函数中初始化 struct kmem_cache 结构中的核心属性。kmem_cache 结构的问题就解决了，但是这两个 slab 缓存池 中的 kmem_cache_node 结构的问题还未解决。

所以内核决定首先创建 boot_kmem_cache_node，并通过 early_kmem_cache_node_alloc 函数为 boot_kmem_cache_node 创建 kmem_cache_node 结构。

当 boot_kmem_cache_node 被创建出来之后，内核就可以动态的分配 kmem_cache_node 对象了。

所以最后创建 boot_kmem_cache，在遇到 kmem_cache_node 结构创建的时候，直接使用 boot_kmem_cache_node 进行动态创建。

最后通过 bootstrap 将这两个临时静态的 slab cache : boot_kmem_cache，boot_kmem_cache_node 深拷贝到最终的全局 slab cache 中。

最终就得到了两个去全局的 slab 缓存池，一个用来管理 struct kmem_cache 类型的对象，一个用来管理 struct kmem_cache_node 类型的对象。